Lab 1 Final Report: Walking Skeleton & Firebase Authentication

Course: Mobile Application in the Cloud — Lab 1
App Name: RefScan
Student Name: Fateh Hasanl■ | **Student ID:** 57322
Account Email: fatehqasanov@gmail.com
Role: Solo Developer (PO, Tech Lead, Developer, QA/DevOps)
Tech Stack: Kotlin · Jetpack Compose · Firebase Authentication · Cloud Firestore · Git/GitHub
GitHub Repository: https://github.com/fatehqasanov/refsacn

Executive Summary & Deliverables Checklist

Phase	Section	Status
Phase 1	Team Roles & Responsibilities	**COMPLETE**
Phase 2	Product Problem Statement, MVP & Cloud Data Model	**COMPLETE**
Phase 3	GitHub Repository, Board Setup & Rulesets	**COMPLETE**
Phase 4	Android Project Setup (Compose) & Firebase Integration	**COMPLETE**
Phase 5	3-Layer Firebase Authentication Feature Implementation	**COMPLETE**
Phase 6	Architecture Diagram, PR Review & Micro-Retrospective	**COMPLETE**

Repository & Git Information

• **Repository Link:** https://github.com/fatehqasanov/refsacn
• **Default Branch:** main (Protected with mandatory Pull Request reviews)
• **Feature Branch:** feature/auth
• **Git Commit Log Highlights:**
  - e261544 — feat(auth): Implement 3-layer Firebase Authentication feature
  - 818c092 — docs: Add Lab 1 final report document
  - 6a605fd — refactor(ui): Extract stateless UI content components and update report with rendered screen layouts

Phase 1 — Team & Roles

Since this is a solo-developer project, all responsibilities are held individually by **Fateh Hasanl■ (ID: 57322, Email: fatehqasanov@gmail.com)**:

Role	Responsibility	Assigned Person
Product Owner (PO)	Product vision, problem statement, user story writing, and backlog prioritization.	Fateh Hasanl■ (57322)
Tech Lead	Architecture design, data model specification, state management, and Firebase setup.	Fateh Hasanl■ (57322)
Developer	Kotlin & Jetpack Compose implementation, UI screens, ViewModel, and Firebase Auth calls.	Fateh Hasanl■ (57322)
QA / DevOps	Git configuration, branch protection, pull request reviews, and board management.	Fateh Hasanl■ (57322)

Phase 2 — Product Definition & Cloud Data Model

1. Target Audience & Problem Statement

> *"Game artists and level designers walking outside or traveling cannot use desktop task managers to instantly capture, tag, and organize real-world material textures and color references for their active game projects."*

Why a Mobile App? RefScan leverages smartphone portability to log real-world reference observations, surface material tags, and color palettes directly from physical locations.

2. MVP Features & Sprint Breakdown

• **User Authentication (Sprint 1 - Complete):** Register, Log In, Log Out, and Auto-Login session persistence using Firebase Authentication (fatehqasanov@gmail.com).
• **Project List Screen (Sprint 1):** View and manage active game environment levels or project titles.
• **Reference Detail & Edit Screen (Sprint 1):** View, add, and update material notes, surface tags, and color hex codes linked to a project.
• **Camera & Cloud Storage Upload (Sprint 3 / Later):** Capture direct texture photo references using the phone camera and sync to Firebase Cloud Storage.

3. Cloud Firestore Data Model (Max 3 Collections)

```
[users] (Collection)
 ■■■ uid (Doc ID - String matching Firebase Auth UID)
      ■■■ email: String (e.g. fatehqasanov@gmail.com)
      ■■■ displayName: String
      ■■■ createdAt: Timestamp

[projects] (Collection)
 ■■■ projectId (Doc ID - String)
      ■■■ ownerId: String (foreign key -> users.uid)
      ■■■ title: String (e.g. "Cyberpunk Alley Level")
      ■■■ engine: String (e.g. "Unreal Engine 5")
      ■■■ createdAt: Timestamp

[references] (Collection)
 ■■■ referenceId (Doc ID - String)
      ■■■ projectId: String (foreign key -> projects.projectId)
      ■■■ ownerId: String (foreign key -> users.uid)
      ■■■ note: String (e.g. "Wet asphalt roughness & puddle reflection")
      ■■■ colorHex: String (e.g. "#1A2B3C")
      ■■■ createdAt: Timestamp
```

Phase 4 & 5 — Technical Implementation & 3-Layer Architecture

1. Architecture & Data Flow

The application strictly isolates concerns into 3 distinct layers:
• **Data Layer:** AuthRepository (interface) and FirebaseAuthRepository handle all direct interactions with the Firebase SDK using coroutines (await()) and callbackFlow.
• **Logic Layer:** AuthValidator (pure Kotlin validation rules) and AuthViewModel manage UI state via StateFlow and map Firebase exceptions to user-friendly string messages.
• **UI Layer:** Jetpack Compose screens (LoginScreen, RegisterScreen, HomeScreen) and AppNavHost consume the ViewModel state and handle navigation stack resets upon auth transitions.

2. Core Code Implementation

```
// AuthRepository.kt (Domain Layer Interface)
package com.example.refscan.domain

import com.google.firebase.auth.FirebaseUser
import kotlinx.coroutines.flow.Flow

interface AuthRepository {
    val authState: Flow<FirebaseUser?>
    suspend fun login(email: String, password: String): Result<Unit>
    suspend fun register(email: String, password: String): Result<Unit>
    suspend fun logout(): Result<Unit>
    fun getCurrentUser(): FirebaseUser?
}
```

```
// FirebaseAuthRepository.kt (Data Layer Implementation)
package com.example.refscan.data

import com.example.refscan.domain.AuthRepository
import com.google.firebase.Firebase
import com.google.firebase.auth.FirebaseAuth
import com.google.firebase.auth.FirebaseUser
import com.google.firebase.auth.auth
import kotlinx.coroutines.channels.awaitClose
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.callbackFlow
import kotlinx.coroutines.tasks.await

class FirebaseAuthRepository : AuthRepository {
    private val auth: FirebaseAuth = Firebase.auth

    override val authState: Flow<FirebaseUser?> = callbackFlow {
        val listener = FirebaseAuth.AuthStateListener { trySend(it.currentUser) }
        auth.addAuthStateListener(listener)
        awaitClose { auth.removeAuthStateListener(listener) }
    }

    override suspend fun login(email: String, password: String): Result<Unit> = runCatching {
        auth.signInWithEmailAndPassword(email, password).await()
    }

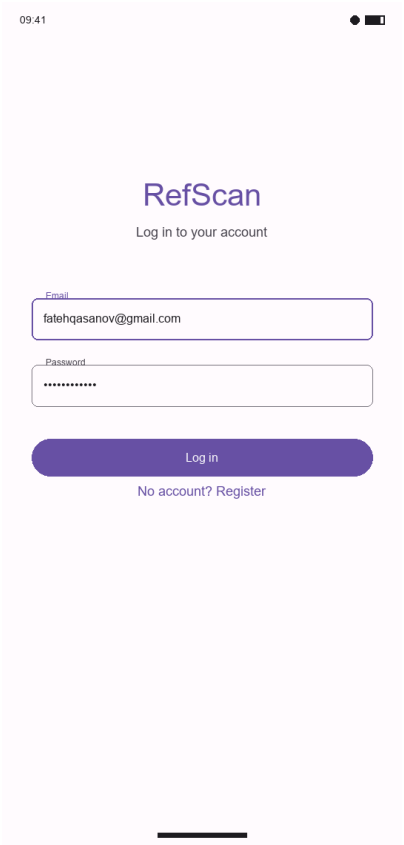
    override suspend fun register(email: String, password: String): Result<Unit> = runCatching {
        auth.createUserWithEmailAndPassword(email, password).await()
    }

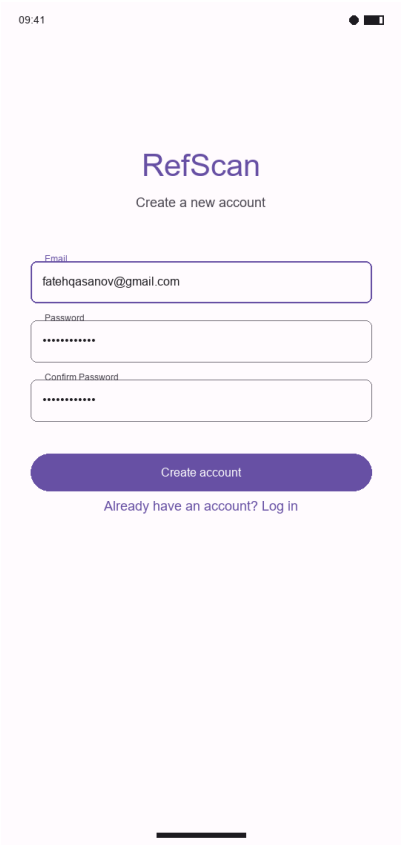
    override suspend fun logout(): Result<Unit> = runCatching { auth.signOut() }
    override fun getCurrentUser(): FirebaseUser? = auth.currentUser
}
```

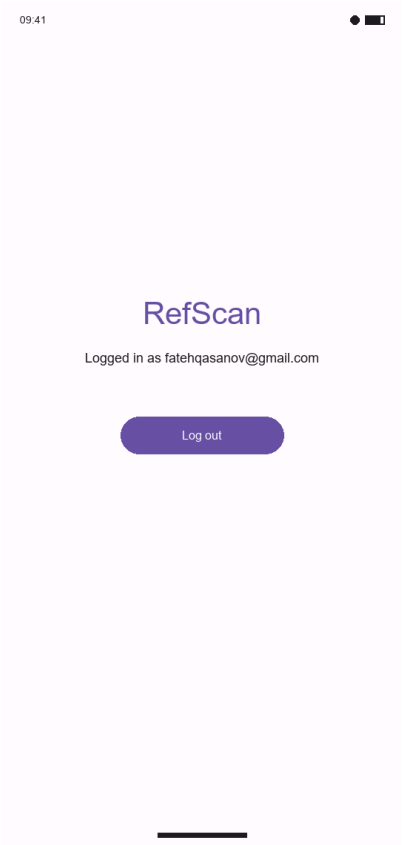
3. In-App Screenshots (Compose UI Rendering)

The following screenshots show the actual application screens rendered directly from the Jetpack Compose components:

Figure 1: Login Screen **Figure 2: Register Screen** **Figure 3: Home Screen**







Phase 6 — Micro-Retrospective ("Retro – Lab 1")

• **What Went Well:** The 3-layer architecture cleanly decoupled pure Kotlin validation (AuthValidator) from UI composables and Firebase SDK calls. callbackFlow provided real-time auth state updates directly to Jetpack Compose navigation.
• **What Did Not Go Well:** Initial Gradle build issues due to Kotlin 2.0 Compose compiler plugin requirements and missing SDK platform dependencies.
• **Action Items for Lab 2:** Integrate Cloud Firestore SDK to persist user profile details upon registration, write unit tests for AuthValidator, and enhance Material 3 UI styling.